

Effortless 3D with react-three-fiber

12/9/2025, 9:14:59 PM

1/26 Effortless 3D with react-three-fiber

Effortless 3D with react-three-fiber

3/26 The WOW effect

The wow effect.

[PAUSE]

This is the moment the user says: 'This is great!' It is never about your biggest feature. It's about UI polish.

It comes from small things:

- A micro-interaction
- A smooth transition
- Some illustrations



These small details create magic.

Why? Because polish proves to the user that we care about their experience. That caring builds trust.

4/26 3D

3D is one of the fastest ways to create that wow effect. It's visual, it's interactive, and it immediately grabs attention. Even a simple 3D touch can make your product stand out

5/26 3D is for everyone

But 3D isn't just for experts – it's a creative tool anyone can use. In this talk, I'll convince you that building immersive web experiences is within your reach.

6/26 About me

My name is Nir. Nir Tamir. I've been doing frontend for over a decade. You can find more about me at nirtamir.com.

7/26 Three.js

We are going to use Three.js - which is a JavaScript library for creating 3D graphics in the browser.

8/26 Three.js structure

Most Three.js apps have the same core elements.

First we need have a **Renderer** that handles rendering your 3D scene in the browser using WebGL (a low-level graphics API built into browsers). (or WebGPU).

Then inside the render we have a **Scene** - which is like the stage to play - it's the space where all your 3D objects, lights, and cameras live.

The **Camera** is like the eyes of the viewer – it defines what part of the 3D scene is visible and how it's projected onto the 2D screen.

Then we can have **Light** - so we won't have everything dark

and we have **Meshes** - which are the 3D object you actually see

9/26 Three.js Meshe

Mesh is made of:

+ **Geometry** (the shape) - It defines the vertices (points in space), faces (triangles built from those points) and the overall shape (cube, sphere, model...)

🎨 **Material** (the look)

This is the "skin" that wraps the geometry.

It controls: • Color • Shininess • Transparency • Texture maps • How it reacts to light

10/26 Three.js Geometries

Here are some geometries with the same normal material. Cube Sphere Cylinder Torus Plane - Notice that they all ends with Geometry

11/26 Three.js Materials

Here on the left we have some materials that does not react to light and on the right it do affected by light. I colored the materials in green color.

13/26 Three.js code

So let's see what vanilla Three.js code looks like.  1 First, we create a WebGLRenderer - this creates a canvas element and handles all the low-level WebGL rendering.  2 Then we create a Scene - think of it as a container for everything in our 3D world.  3 We add a PerspectiveCamera to define our viewpoint.  4 Finally, we create a mesh by combining a BoxGeometry with a MeshBasicMaterial.  5 Then we need to add the mesh to the scene  6 and render it with our camera  7 Notice how verbose this is - we'll see how R3F simplifies this in a moment.

14/26 **React-three-fiber**

So we've seen Three.js - it's powerful but verbose. This is where react-three-fiber comes in. R3F is a React renderer for Three.js - which means it uses React's reconciliation to manage your 3D scene. Instead of imperatively creating objects and adding them to the scene, you declare what you want using JSX. It handles all the setup boilerplate and gives you React's component model for free.

Here's the same example, but written with React Three Fiber.

Inside it, we add `<OrbitControls />` so the user can rotate and zoom with the mouse.

Then we create a with a box geometry and a Matcap material, just like in Three.js, but here we write it as JSX.

[move]

I've added an axesHelper - the red line is X, green is Y, blue is Z. Notice the mesh has position, scale, and rotation properties. Position moves it in 3D space - [x, y, z]. Scale changes its size - [1, 1, 1] is normal size. Rotation turns it - measured in radians, not degrees. Play with these values to get a feel for 3D coordinates.

🔗 1

Now let's understand 3D space better. I've added an axesHelper - the red line is X, green is Y, blue is Z. Notice the mesh has position, scale, and rotation properties. Position moves it in 3D space - [x, y, z]. Scale changes its size - [1, 1, 1] is normal size. Rotation turns it - measured in radians, not degrees. Play with these values to get a feel for 3D coordinates.

🔗 2

Now we're seeing React's real power. I've extracted the box into its own component. This lets us reuse it - see how we render two boxes at different positions? The Box component also has state - it tracks hover and active states. Click a box to scale it up. Hover to change its color. This is the magic of R3F - 3D objects behave like React components with props, state, and events.

🔗 3

Let's add animation. `useFrame` is R3F's animation hook - it runs every frame, about 60 times per second. We get `delta`, which is the time since the last frame. By rotating the box by `delta` each frame, we get smooth, frame-rate-independent animation. The `ref` gives us access to the actual Three.js mesh object. This is how you create any animation in R3F - update values in `useFrame`.

18/26 So how do I use GLTF in React?

Of course, you don't have to model everything from primitives. You can import models created in Blender, Maya, or other 3D software. The standard format is GLTF - think of it as the JPEG of 3D. The gltfjsx tool converts GLTF files into React components automatically. It parses the model and generates code with all the meshes, materials, and hierarchies. Visit gltf.pmnd.rs to try it yourself - drag any GLTF file and get React code instantly.

להציג מודל נעל

19/26 Physics

Now for something really cool - physics. React-three/rapier brings realistic physics to your 3D scenes. Wrap objects in RigidBody components and they become physical objects. They'll fall, collide, bounce - all automatically.

🔗 1

The colliders property tells Rapier the shape for collision detection. 'cuboid' for boxes, 'ball' for spheres, 'trimesh' for complex meshes. The debug mode shows the collision shapes - turn it off for production. Watch what happens when these objects fall and hit the ground.

🔗 2

Now if we change the Sphere collider to from ball to cuboid the physics engine will treat it like a cube. (DEMO)

21/26 The best way to
 learn is by doing

Start small - make a rotating cube. [TODO: slide of just a box] Add some interaction. [TODO: slide of just a box rotating] Import a model and before you know it, you'll be creating those wow moments. [TODO: slide of just a rubik cube rotating]

22/26 Demo wow

This is a model of myself that I built, using the same principles

Here's a more complete example of what you can build. This uses everything we've covered - models, materials, physics, interactivity. The key insight is that each of these effects is just a few lines of code. You don't need to be a graphics programmer or a 3D artist. You just need to understand the building blocks we've covered today.

23/26 Examples

You can also look at the official react-three-fiber docs - they have a lot of impressive examples you can grab and look at their code implementation

25/26

Thank you for your time! The slides are available at nirtamir.com I'm happy to answer questions after the talk or you can reach me through my website.